

Design and Development

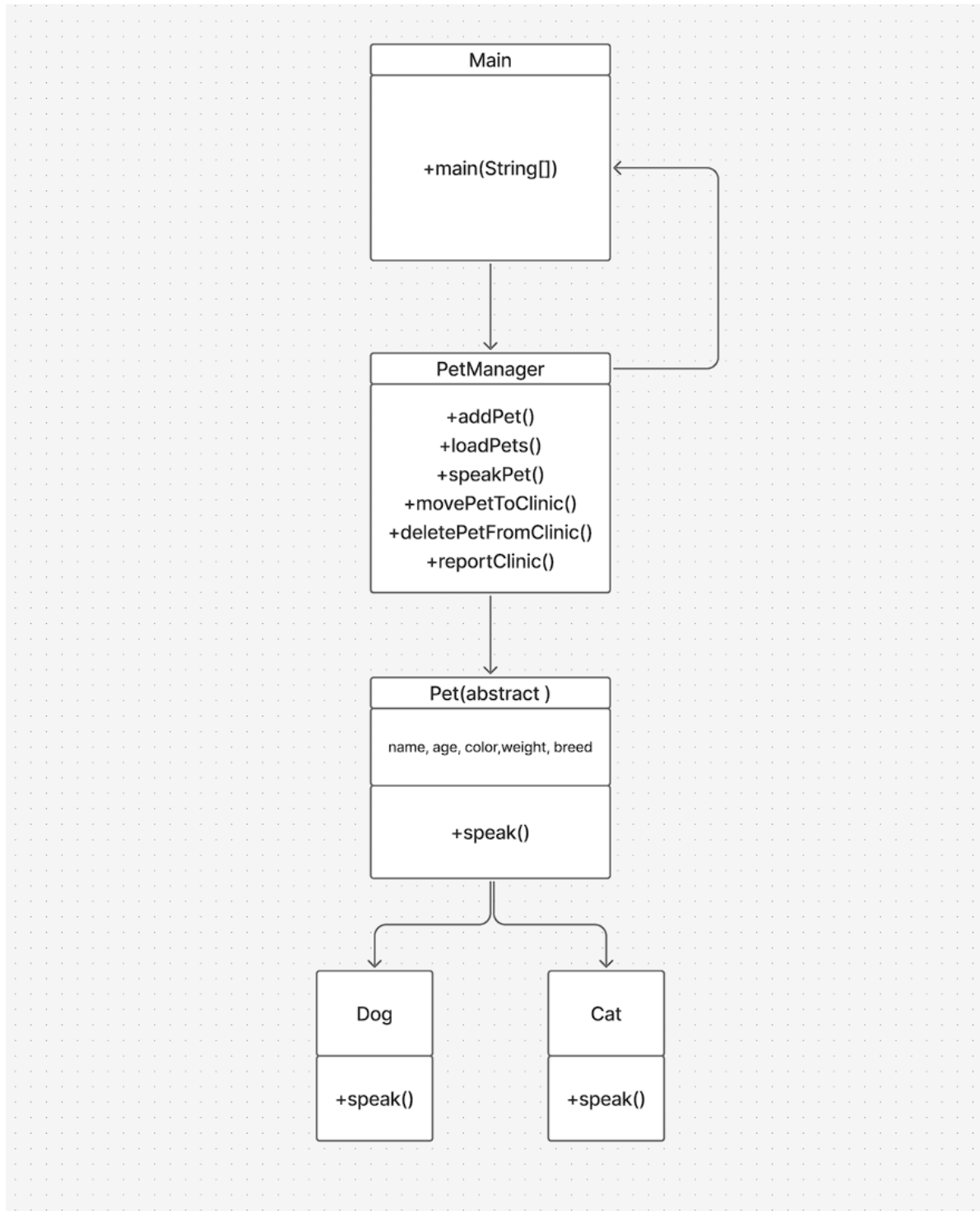
Artom Kovalenko B00974470

Table of Contents

UML Diagram.....	2
Inheritance hierarchy design.....	3
Polymorphism Demonstration	3

UML Diagram

The application follows the object-oriented design centered around inheritance hierarchy. Below is a representation of the UML structure.



Inheritance hierarchy design

The Pet class is an abstract entity that encapsulates common attributes and behaviours. The Dog and Cat classes inherit from Pet, each overriding the speak() method. This design promotes code reuse and facilitates clear extensibility.

Polymorphism Demonstration

Pet references are utilized throughout the application to manage Dog and Cat objects polymorphically, particularly within the ClinicManager and Main classes.

Data Structures and Control Structures

Data Structures

- List: Utilized for storing collections of Pet objects.
- File I/O: Pets are serialized and deserialized via CSV-formatted text files.
- String arrays: Temporarily employed for parsing CSV data.

Program Control Structures

- Loops: Applied for iterating over pets and handling menu logic.
- Conditionals: if/switch statements are used for branching logic in menu operations.
- Exception handling: try-catch blocks ensure effective I/O error management.

Main Application

The entry point is the Main class, which uses a while(true) loop with a menu-driven interface. It interacts with the ClinicManager, creates pets, and interprets user input. By using the overridden speak() methods and generating a Dog or Cat depending on user input, it illustrates dynamic dispatch.

Design Summary

Data (Pet), logic (ClinicManager), and user interface (Main) are divided into a modular architecture. OOP approaches improve scalability and maintainability.

During development, text files offer simple and transparent storage. Reporting and extensive CRUD operations are supported via the console-based user interface.